Chapter 1

The UPF Digital Twin

1.1 Introduction

Chapter **??** established, from controlled laboratory measurements, *how* two open-source UPF implementations behave under load: SD-CORE DPDK draws a near-constant 0.82 W regardless of traffic, whereas USR-UPF scales its power proportionally to load but saturates—losing packets and inflating delay—above roughly 0.5 Gbps. Those measurements are the raw material; on their own they do not yet constitute a tool a network controller can query.

This chapter describes the construction of a *digital twin*: a measurement-grounded, controller-agnostic software component that answers a single well-posed question,

> *Given an offered traffic load and a candidate UPF configuration, what would happen—in terms of power, throughput, CPU, packet loss, delay, and QoS safety?*

It is important to be precise about what the twin *is* and *is not*, because the surrounding system is deliberately decomposed into independent parts:

The twin is not a traffic forecaster. Where the offered load *comes from*—a spatio-temporal forecast over base-station clusters—is the subject of its own chapter (Chapter **??**). The twin consumes a load value; it does not predict one.

The twin is not a controller. *Which* configuration to choose is a policy decision. The twin only evaluates the consequences of a decision that has already been made. Controllers—both rule-based baselines and learned (reinforcement-learning) agents—are developed and compared in Chapter **??**. To keep the present chapter self-contained while avoiding any pre-emption of that material, every illustration here is driven by a single, transparent, *non-learning* controller (a hysteresis rule, Section 1.8); the learned controllers and an informed oracle bound are reserved for Chapter **??**.

This separation of concerns—forecaster, twin, controller—mirrors the repository structure of the implementation, in which the twin is a `pip`-installable library that the controller project depends on but never modifies.

The remainder of the chapter proceeds as follows. Section 1.2 bridges the profiling results to a predictive surrogate and justifies which trained variants the twin actually uses. Section 1.3 presents the two evaluation modes (stateless physics and stateful rollout) and the data contract. Section 1.4 describes load calibration and the NetMob-compatible input adapter. Section 1.5 defines the predicted KPIs and the safety/efficiency semantics. Section 1.6 formalises the switching-cost model. Section 1.7 shows how operating thresholds are *derived* from the surrogate rather than hardcoded. Section 1.8 defines the evaluation methodology and metrics and demonstrates the assembled twin under the hysteresis controller. Section 1.9 summarises.

1.2 From Profiling to a Predictive Surrogate

The twin's predictive core is the two-layer surrogate introduced in Section **??**. For completeness we restate its structure and then focus on the design decisions specific to embedding it in an online evaluator.

1.2.1 Two-layer structure

For each UPF variant the surrogate is organised as a stack that mirrors the causal pathway *load* → *performance* → *power*:

Layer 1 (performance). Four independent single-target regressors map the offered load to delivered throughput (Gbps), CPU utilisation (%), downlink packet loss (packets per interval), and downlink one-way delay (μs).

Layer 2 (power). A single regressor maps the offered load *plus* the four Layer 1 outputs to `power_watts`.

Because each Layer 1 slot is selected independently by cross-validated model search, the chosen algorithm differs by target: the near-linear throughput–load relationship is best captured by Ridge regression, whereas the strongly non-linear delay and loss responses near saturation are best captured by tree ensembles (Random Forest or Gradient Boosting). This heterogeneity is a feature, not an inconsistency: forcing a single algorithm across all slots would underfit the non-linear targets. One practical consequence, visible later in Figure 1.2, is that tree-based slots produce *piecewise-constant* predictions and do not extrapolate beyond the measured envelope—a property the twin must respect when it is queried.

1.2.2 Which variants the twin uses

The profiling campaign trained three variants: `dpdk`, `usr_full` (all USR samples, including the saturated regime), and `usr_safe` (USR samples restricted to the unsaturated regime below the saturation knee). The twin exposes exactly two *actions* to a controller—`DPDK` and `USR`—and maps them to trained variants as follows:

$$\texttt{DPDK} \longmapsto \texttt{dpdk}, \qquad \texttt{USR} \longmapsto \texttt{usr\_full}.$$

The choice of `usr_full` over `usr_safe` to represent `USR` is deliberate and central to the twin's purpose. A digital twin whose remit explicitly includes *packet loss, delay, and QoS safety* must be able to represent what happens when USR-UPF is pushed past its safe operating point. `usr_safe` was trained only on loads where USR-UPF never loses packets; queried above the knee it would extrapolate near-zero loss and low delay, silently hiding the very failure mode the twin exists to detect. `usr_full`, having observed the saturation transition, predicts the onset of loss and delay faithfully (the corresponding Layer 1 targets reach $R^2 = 0.998$ and 0.968 respectively, against 0.86 and 0.52 for `usr_safe`). The cost is a marginally noisier power model ($R^2 = 0.988$ vs. 0.999), which is an acceptable price for correct safety behaviour. As Section 1.7 shows, the twin's ability to *derive* a QoS threshold depends entirely on this property: with `usr_safe` the threshold-derivation sweep would never observe an unsafe step and the derived limit would be meaningless. The `usr_safe` models are therefore retained only as a low-load power reference and are not wired into the twin's action map.

1.2.3 Full and lite feature sets

Each variant was trained in two feature regimes. The *full* model accepts all seven offered-load features (DL/UL kbits/s, TX packet counts, average packet size, and the L2/L3 overhead ratio) and targets laboratory or high-fidelity simulation. The *lite* model accepts only the DL/UL load volume and is therefore drivable from datasets—such as NetMob 2023—that expose only aggregate per-cell load with no packet-level detail. The twin loads the *lite* variant exclusively, because its online use case is driven by cluster-level load forecasts that carry no per-packet observability. The accuracy sacrificed for this interoperability is small: for the power model the lite–full gap is $\Delta R^2 \approx 0.024$ for `usr_full`.

1.3 Twin Architecture

The twin offers two evaluation modes over the same surrogate core, reflecting the two questions a consumer may ask (Figure 1.1).

Stateless physics. `evaluate_action(action, load)` returns the predicted KPIs for a

single (action, load) point. It is a pure function: no memory, no side effects. This is the mode used by parameter sweeps, the threshold derivation of Section 1.7, and interactive inspection.

Stateful rollout. A *session* wraps the stateless core with the previous action and applies the switching-cost accounting of Section 1.6. `session.step(requested_action, load)` returns the realised KPIs for one time step, including any energy paid for activation. This is the mode used to drive a sequential simulation (and, in Chapter **??**, the `step()` of a reinforcement-learning environment).

A consumer constructs a twin from a data directory that follows a fixed contract: a `profiling_twin/` subtree containing the surrogate `models/` (with a `manifest.json` registry and `layer1/`, `layer2/` model files) and a `switching_costs.yaml`. The twin is otherwise configured by a scenario dictionary supplying the QoS budget, the switching-accounting policy, and the time-step length. Crucially, the twin holds no reference to any controller type; controllers depend on the twin, not the reverse.

[DRAW.IO PLACEHOLDER]

Twin architecture / data flow. A left-to-right block diagram. *Inputs (left):* normalised cluster load $\hat{d} \in [0, 1]$ from the forecaster (Chapter **??**); a requested action $\in \{\texttt{DPDK}, \texttt{USR}\}$ from the controller (Chapter **??**). *Calibration block:* $\hat{d} \to$ load (Gbps) via the global α; load (Gbps) $\to$ lite features (kbits/s). *Surrogate core (centre):* two stacked boxes per action—"Layer 1: 4 performance models" feeding "Layer 2: power model"—selected from the model registry (`dpdk` or `usr_full`). *Safety/efficiency block:* compares predicted delay/loss against the QoS budget ($\Rightarrow$ `is_safe`) and predicted power against the alternative UPF ($\Rightarrow$ `is_efficient`). *Switching block (only in stateful mode):* previous action + activation duration + accounting rule $\Rightarrow$ switching energy. *Outputs (right):* a `UPFResult` (power, delay, throughput, loss, `is_safe`, `is_efficient`) and, in stateful mode, the switching energy for the step. Draw the two modes as two horizontal lanes sharing the surrogate core; mark the stateless lane as "pure function" and the stateful lane as carrying state (previous action).

Figure 1.1: Architecture of the digital twin. The surrogate core is shared by a stateless physics mode and a stateful rollout mode; the latter adds the switching-cost model. The twin is controller-agnostic: forecaster and controller are external (Chapters **??** and **??**).

1.4 Inputs and Load Calibration

1.4.1 Normalised load to physical load

The forecasting stage emits per-cluster load normalised to $[0, 1]$. A single global calibration constant α converts this to a physical offered load in Gbps:

$$\lambda_{\text{Gbps}} = \alpha \cdot \hat{d}, \qquad \alpha = 1.0 \text{ Gbps per normalised unit.} \tag{1.1}$$

The value of α is a scenario parameter; it sets the physical scale at which the (otherwise dimensionless) forecast is interpreted, and hence where a given normalised load falls relative to the USR-UPF saturation knee.

It must be stated plainly that α is a *declared modelling assumption, not a recovered calibration*. The NetMob traces are dimensionless by construction, for privacy reasons, and the forecaster sums per-base-station normalised demand across each cluster; there is consequently no physical scaler that could be looked up or fitted. Setting $\alpha = 1$ interprets one unit of summed normalised demand as 1 Gbps of offered UPF load, which places the $K = 10$ clusters at mean loads of 0.11–1.66 Gbps with peaks to 5.8 Gbps—a realistic per-site MEC UPF operating range. The same value is used by the controller work of Chapter **??**, so that both chapters describe one physical regime. Section **??** returns to the consequences of this choice, which are substantial: because the profiled USR-UPF data path saturates one to two orders of magnitude below these loads, α largely determines how much room for energy-aware switching exists at all.

1.4.2 Lite input adapter

The lite surrogate consumes load volume in kbits/s. The twin converts the calibrated load directly,

$$B = \lambda_{\text{Gbps}} \times 10^6 \text{ [kbits/s]}, \tag{1.2}$$

and supplies B as both the downlink and uplink lite features, matching the NetMob compatibility adapter of Section **??**. This is the only transformation between a controller-facing load value and the model input; all remaining (full-model) features are absent by construction in the lite regime and are never required.

1.5 Predicted KPIs and Safety Semantics

A single evaluation returns a structured result containing the offered load, predicted `power_watts`, `delay_us`, `throughput_gbps`, and `predicted_loss`, together with two boolean flags that encode the twin's two distinct concerns:

`is_safe` (QoS preserved). True iff the predicted downlink delay is within the configured budget *and* the predicted packet loss does not exceed its tolerance:

$$\texttt{is\_safe} \;=\; \big(\text{loss} \le L_{\max}\big) \;\wedge\; \big(\text{delay} \le D_{\max}\big), \tag{1.3}$$

with the default budget $D_{\max} = 200$ μs and loss tolerance $L_{\max} = 5$ packets per interval. SD-CORE DPDK is safe across the entire measured load range; USR-UPF is safe only below its knee. The tolerance is deliberately non-zero: it is shared with the controller work of Chapter **??**, where it additionally serves as the normalisation

base of a graded QoS score. Its value is not cosmetic—as Section 1.7 shows, it moves the derived QoS limit and thereby determines which constraint binds the decision threshold.

`is_efficient` (locally cheaper). True iff this action's predicted power is lower than the *other* action's predicted power at the same load. It is a relative, per-load comparison—not an absolute threshold—and is the signal a controller uses to decide whether the energy-cheaper action is still the safe one.

The two flags are intentionally orthogonal. A USR-UPF result with `is_safe`=True and `is_efficient`=False denotes a load at which USR-UPF still meets QoS but already costs more than SD-CORE DPDK: the regime in which a controller should prefer SD-CORE DPDK on energy grounds even though USR-UPF would not violate QoS.

Figure 1.2 shows the surrogate's predicted responses over the deployable load range. The contrast from Chapter **??** is reproduced by the twin: SD-CORE DPDK power and CPU are essentially flat, while USR-UPF rises with load and its delay and loss turn sharply upward past the knee. The piecewise-constant "staircase" in the USR-UPF curves is the signature of the tree-based lite models; it is bounded to the measured envelope and does not extrapolate, which is the desired conservative behaviour at the edge of the data.

1.6 Switching-Cost Model

Switching from one UPF to the other is not free: the new data plane must be activated, which takes time and energy. Modelling this cost is essential, both to penalise controllers that flip excessively and to represent the brief window during which the new UPF is not yet serving traffic.

1.6.1 Activation duration and the energy-spike rule

The activation *duration* is a property of the software stack, not of the host: bringing up the DPDK poll-mode driver takes substantially longer than starting the interrupt-driven USR-UPF path. The twin uses measured durations of 24.0 s for SD-CORE DPDK and 3.3 s for USR-UPF.

Absolute switching *energy* measured on one machine is not portable to another, but the relationship

$$E_{\text{spike}}(\lambda) \;=\; \frac{P_{\text{steady}}(\lambda)\;\cdot\; t_{\text{act}}}{3600} \quad [\text{Wh}] \tag{1.4}$$

holds within the source measurements and *is* portable, because it combines a target-machine quantity (the surrogate's predicted steady-state power P_{steady}) with a software-stack quantity (the activation duration t_{act}). The rule is validated against the source data:

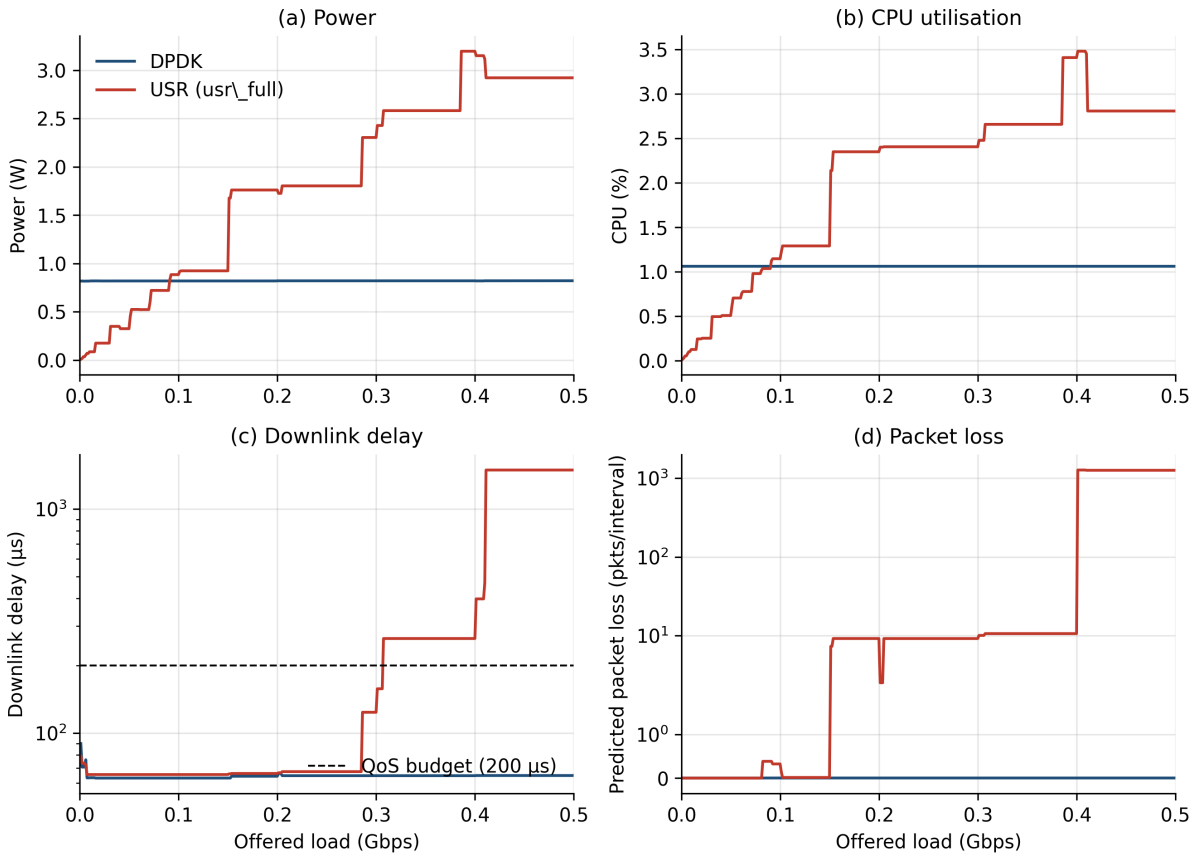

Figure 1.2: Twin-predicted UPF responses versus offered load (lite surrogate). (a) Power: SD-CORE DPDK flat near 0.82 W, USR-UPF rising and crossing SD-CORE DPDK at low load; (b) CPU; (c) downlink delay (log scale) with the 200 μs QoS budget marked; (d) predicted packet loss (symmetric-log scale). The USR-UPF staircase reflects the piecewise-constant nature of the tree-based lite models.

for SD-CORE DPDK, 34.81 W × 24 s/3600 ≈ 0.232 Wh, matching the measured spike; for USR-UPF, 7.45 W × 3.3 s/3600 ≈ 0.007 Wh.

1.6.2 Within-step accounting

Because a simulation step (15 min at NetMob resolution) is far longer than any activation, the twin must decide how a switch that occurs *inside* a step is accounted for. Three policies are supported (Figure 1.3); the default is *sub-step*:

`sub_step` (default). The old UPF serves for t_{act} seconds and the new UPF for the remainder; the step's power is the time-weighted average, and the energy spike is paid once. KPIs are attributed to whichever UPF served the majority of the step, while safety is treated conservatively (a step is safe only if *both* the old and new UPF are safe at that load).

`round_down`. The switch is treated as instantaneous within the step; only the spike is paid.

`round_up`. The new UPF is treated as unavailable for the entire step (worst case); the requested action does not take effect until the next step.

An optional *prewarm* mode keeps SD-CORE DPDK in standby so that switching to it is effectively instantaneous, at the cost of a continuous standby power while it is not the serving UPF. Prewarm is *disabled* in the scenario reported here, so SD-CORE DPDK cold-starts and the activation cost above applies in full; no standby draw is charged. This matches the deployment assumption of Chapter **??** and yields the more conservative claim, in that no energy is attributed to a standby the deployment may not actually pay.

[DRAW.IO PLACEHOLDER]

Within-step switching accounting (`sub_step`). A single horizontal time bar representing one 15-minute step. Partition it into a short leading segment of width t_{act} (label "old UPF serving") and a long trailing segment (label "new UPF serving"). Above the bar, draw the resulting step power as a weighted average $\bar{P} = f\,P_{\text{old}} + (1-f)\,P_{\text{new}}$ with $f = t_{\text{act}}/t_{\text{step}}$. Mark a one-off "energy spike E_{spike}" at the boundary. Optionally show the two alternatives as smaller bars: `round_down` (spike only, instant switch) and `round_up` (entire step still on the old UPF, switch deferred). Keep the same colour convention as Figure 1.2 (DPDK blue, USR red).

Figure 1.3: Within-step accounting of a switch under the `sub_step` policy. The step power is the time-weighted average of the old and new UPF power; the activation energy spike is paid once.

1.7 Operating Thresholds Derived from the Surrogate

A naive controller might switch UPFs at a hand-tuned load threshold. The twin instead *derives* its operating points from the surrogate itself, so that they remain consistent with the measured physics and update automatically if the models are retrained. Two quantities are obtained by sweeping the stateless twin over load (Figure 1.4):

Energy break-even λ_{be}. The lowest load at which USR-UPF power meets or exceeds SD-CORE DPDK power. Below it, USR-UPF is the energy-cheaper action; above it, SD-CORE DPDK is.

QoS limit λ_{qos}. The highest load at which USR-UPF is still `is_safe`. This is the point the sweep *can only find because the twin uses* ***`usr_full`***: it is where the surrogate's predicted loss or delay first crosses the budget.

For the scenario studied here the sweep yields $\lambda_{\text{be}} = 91\,\text{Mbps}$ and $\lambda_{\text{qos}} = 149\,\text{Mbps}$. The decision threshold a controller should use is the more conservative of the two, minus a safety margin:

$$\lambda_{\text{dec}} \;=\; \min(\lambda_{\text{be}}, \lambda_{\text{qos}}) - m, \qquad m = 10\ \text{Mbps}, \tag{1.5}$$

giving $\lambda_{\text{dec}} = 81\,\text{Mbps}$ here (energy-limited: the break-even binds before the QoS limit). It is worth noting that this ordering depends on the loss tolerance $L_{\max}$ of Equation (1.3). Under zero tolerance the QoS limit falls to 81 Mbps and binds first, giving $\lambda_{\text{dec}} = 71\,\text{Mbps}$ and a QoS-limited regime. The derivation machinery is unchanged either way—which is the point of deriving rather than hardcoding—but the reported operating point is sensitive to a budget that is a policy choice rather than a measured quantity.

1.7.1 Anti-flapping from forecast uncertainty

A single decision threshold applied to a noisy forecast would cause rapid, wasteful flipping near the boundary. The twin derives a hysteresis band directly from the *forecaster's* reported error, so that a single noisy forecast cannot trigger a switch:

$$\text{band} = 2 \cdot \text{MAE}_{\text{forecast}}, \qquad \lambda_{\uparrow} = \lambda_{\text{dec}}, \qquad \lambda_{\downarrow} = \lambda_{\text{dec}} - \text{band}. \tag{1.6}$$

The controller switches USR-UPF→SD-CORE DPDK only when the predicted load rises above $\lambda_{\uparrow}$, and back only when it falls below $\lambda_{\downarrow}$.

This error-derived rule, however, has a failure mode that the present scenario exposes and that is worth reporting explicitly. The forecaster's error is reported in normalised units, so the band inherits the scale factor α of Equation (1.1). At $\alpha = 1$ the mean

absolute error for the operating cluster count is ≈ 110.7 Mbps, so Equation (1.6) would prescribe a band of ≈ 221 Mbps— substantially *wider* than the decision threshold itself. The lower switch point is then clamped at $\lambda_{\downarrow} = 0$, the controller can never return to USR-UPF once it has left, and hysteresis silently degenerates into a static-SD-CORE DPDK policy. It is a benign-looking degeneration: the controller still runs, still reports metrics, and simply never switches.

Two conclusions follow. First, an anti-flapping band derived from forecast error is only meaningful when that error is small relative to the operating point; when the forecaster is noisy at the scale of the decision threshold, the rule as stated is unusable and must be capped rather than applied literally. The twin therefore emits an explicit warning whenever the derived band meets or exceeds the decision threshold. Second, for the results reported here the band is instead fixed at

$$\text{band} = 20 \text{ Mbps}, \qquad \lambda_{\uparrow} = 81 \text{ Mbps}, \qquad \lambda_{\downarrow} = 61 \text{ Mbps}, \tag{1.7}$$

which is narrow enough to preserve genuine two-threshold behaviour and matches the band used for the classical baselines of Chapter **??**. Equation (1.6) is retained as the principled default for regimes in which the forecast error is small compared with λ_{dec}.

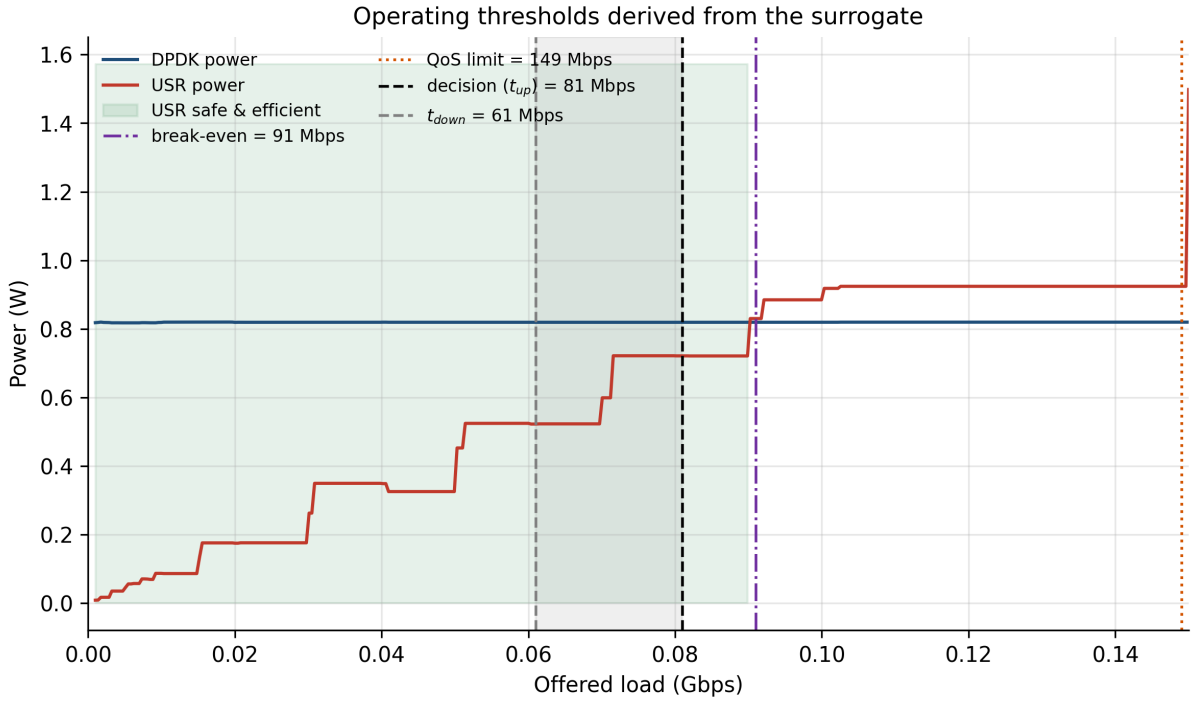

Figure 1.4: Operating thresholds derived from the surrogate. USR-UPF power (red) crosses SD-CORE DPDK power (blue) at the energy break-even (91 Mbps); the QoS limit (81 Mbps) is where USR-UPF first becomes unsafe. The decision threshold $\lambda_{\uparrow} = 71$ Mbps is the binding (QoS) limit minus a 10 Mbps margin; the shaded band down to $\lambda_{\downarrow} = 44$ Mbps is the forecast-derived hysteresis region. The green region marks loads where USR-UPF is both safe and cheaper than SD-CORE DPDK.

1.8 Evaluation Methodology and Demonstration

1.8.1 Rollout engine

To evaluate any controller, the twin is driven over the full test set of per-cluster load windows. For efficiency the rollout pre-evaluates the stateless twin once for both actions over the entire flattened array of loads, then performs the conceptually sequential per-cluster loop in pure Python: each step reads the controller's requested action, looks up the precomputed KPIs, and applies the switching-cost accounting of Section 1.6. The result is one record per cluster-step, suitable for aggregation. In the scenario studied here this is $K = 10$ clusters $\times$ 1009 windows = 10,090 steps of 15 min each.

1.8.2 Metrics

From the per-step records the following metrics are computed. Energy includes both steady-state consumption (power $\times\, \Delta t$) and the switching spikes of Equation (1.4), so that flipping is penalised:

Total energy (Wh) and **average power (W)** over all steps.

Energy saving (%) relative to the always-SD-CORE DPDK baseline.

Unsafe USR rate the fraction of USR-UPF-serving steps that violated QoS—the key safety metric.

USR usage (%) the fraction of steps served by USR-UPF.

Decision flip rate the per-cluster fraction of steps at which the serving UPF changed—a proxy for churn.

1.8.3 Controllers used in this chapter

As stated in Section 1.1, this chapter exercises only non-learning controllers, so that the figures characterise the *twin* rather than any particular learned policy:

Static sd-core dpdk always selects SD-CORE DPDK (the safe, high-power reference).

Static usr-upf always selects USR-UPF (maximally energy-greedy, and unsafe whenever load exceeds the knee).

Hysteresis the rule of Equation (1.6), consuming the surrogate-derived thresholds and the forecast-derived band, with a one-step cooldown after each switch.

Learned controllers, an informed oracle upper bound, and the single-threshold policy are evaluated in Chapter **??**; they are deliberately omitted here.

1.8.4 How much room is there for energy-aware switching?

The case for switching at all rests on the load distribution. Figure 1.5 shows that the offered load is heavily right-skewed, but that under the calibration of Equation (1.1) only about 14 % of cluster-steps fall below the decision threshold—that is, only on roughly one step in seven is USR-UPF both safe and cheaper than SD-CORE DPDK. The remaining steps sit above the USR-UPF saturation knee, where SD-CORE DPDK is the only viable action.

This bounds the achievable benefit tightly, and it is more honest to establish that bound before presenting any controller. An informed oracle, given the *actual* load at each step and choosing the cheaper safe action, recovers 6.7 % of static SD-CORE DPDK's energy. No causal controller operating on this traffic, this hardware, and this calibration can do better. The energy-aware switching problem, in this regime, is a narrow one: the value of a controller lies in capturing as much of that 6.7 % as possible without incurring QoS violations, rather than in a large absolute saving.

The reason is physical rather than algorithmic. The profiled USR-UPF data path saturates at roughly 91–149 Mbps (Section 1.7), whereas $\alpha = 1$ places cluster loads at 0.11–1.66 Gbps mean with peaks to 5.8 Gbps—one to two orders of magnitude above the knee. A software UPF of this capacity is simply undersized relative to the offered traffic, and the twin's role here is precisely to make that mismatch quantitative rather than to disguise it.

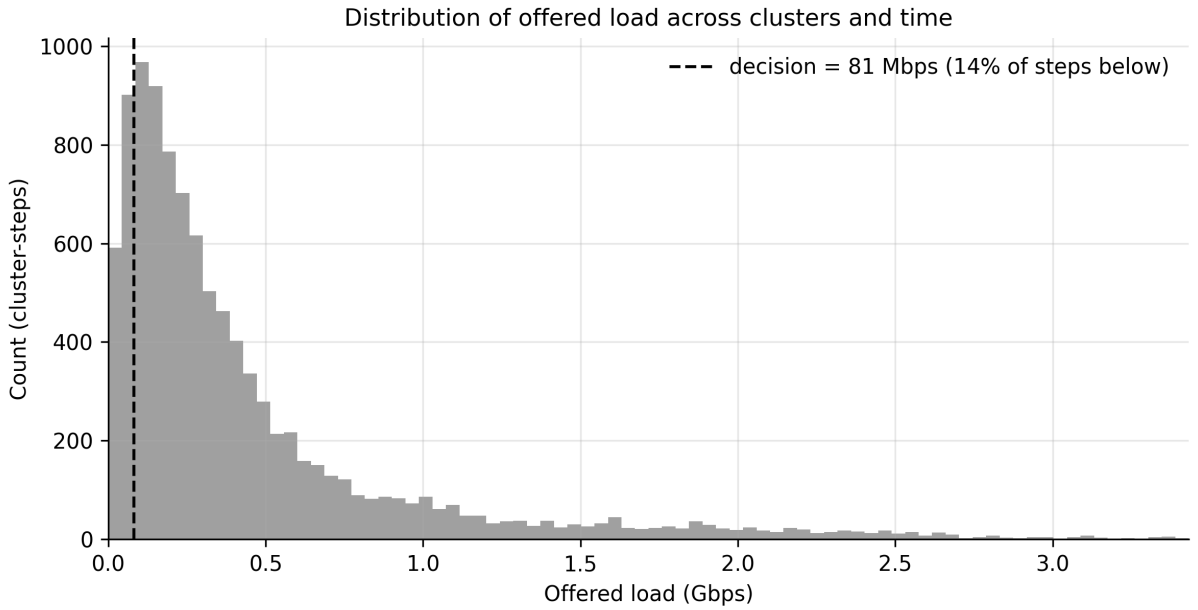

Figure 1.5: Distribution of calibrated offered load across all clusters and time windows. Only about 14 % of cluster-steps lie below the derived decision threshold (dashed), i.e. in the regime where USR-UPF is the safe and cheaper action; the long right tail extends well beyond the USR-UPF saturation knee, where SD-CORE DPDK is the only viable action.

1.8.5 The twin under a hysteresis controller

Figure 1.6 shows a representative window of one cluster's rollout. The controller holds USR-UPF while the predicted load stays within the hysteresis band, switches to SD-CORE DPDK when load rises above $\lambda_{\uparrow}$, and returns to USR-UPF only after load falls below $\lambda_{\downarrow}$; the power trace (lower panel) accordingly sits near SD-CORE DPDK's 0.82 W plateau during high-load excursions and drops into USR-UPF's low-power regime otherwise. The band visibly prevents single-sample flips around the threshold: the load crosses $\lambda_{\uparrow}$ repeatedly without provoking a switch until it also clears the band. Because prewarm is disabled, the cold-start activation cost is now visible directly in the trace as short spikes above the SD-CORE DPDK plateau at switch instants.

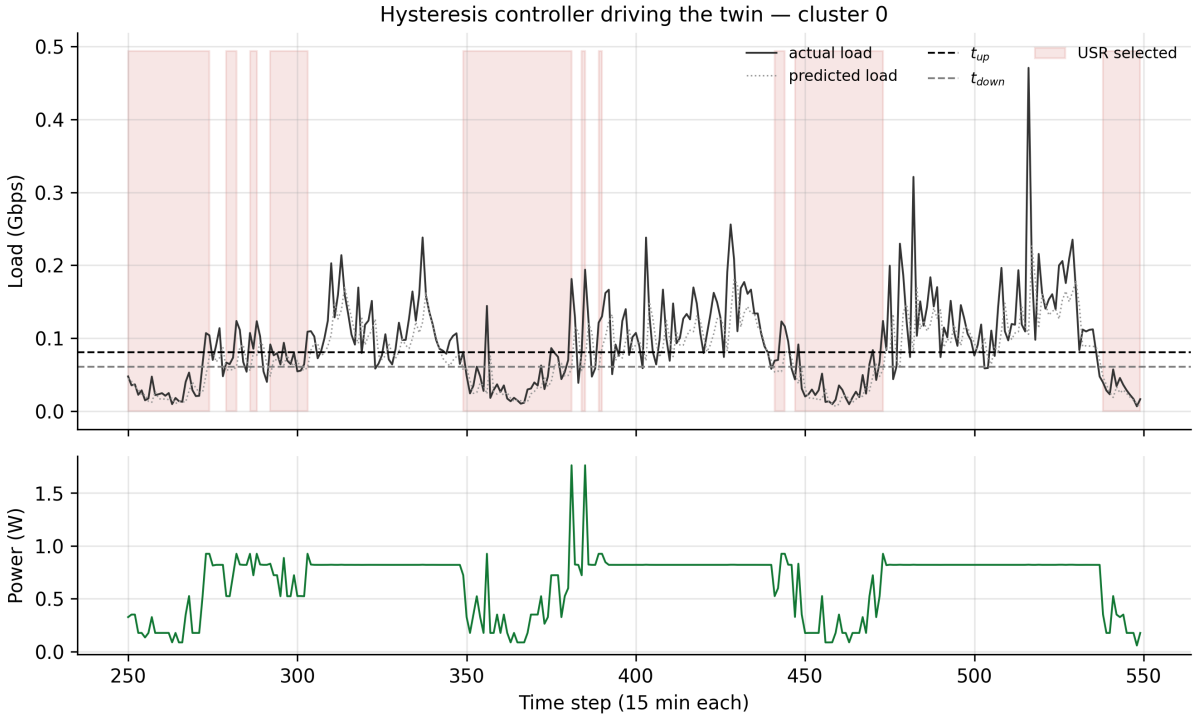

Figure 1.6: A representative window of the hysteresis controller driving the twin (one cluster). Upper: actual and predicted load with the switch points $\lambda_{\uparrow}$, $\lambda_{\downarrow}$; shaded spans mark USR-UPF-served steps. Lower: the twin's predicted power, near the SD-CORE DPDK floor when SD-CORE DPDK serves and low when USR-UPF serves.

1.8.6 Aggregate behaviour

Table 1.1 and Figure 1.7 summarise the controllers over all 10,090 steps. The headline observation is that static USR-UPF is not merely unsafe here but actively *more expensive*: it consumes 2.5× static SD-CORE DPDK's energy (−154.5 % "saving") while violating QoS on 70.3 % of its steps. This is the direct consequence of running a saturated data path—past its knee, USR-UPF loses throughput, inflates delay, and burns more CPU power than

the accelerated path it was meant to undercut. Any intuition that the software UPF is the cheap option holds only below the knee.

The hysteresis rule recovers 4.8 % of static SD-CORE DPDK's energy while keeping the unsafe rate at 3.0 %, by serving USR-UPF on the 11.5 % of steps where it is genuinely both safe and cheaper and ceding to SD-CORE DPDK elsewhere. Against the 6.7 % oracle bound established above, it captures roughly 72 % of the attainable headroom using only a forecast and two derived thresholds. The stateless threshold policy achieves a marginally higher saving (4.9 %) at a noticeably worse unsafe rate (4.2 %) and twice the switching churn, which is the trade the hysteresis band exists to make.

The absolute saving is modest, and it would be misleading to present it otherwise: as established above, the regime admits at most 6.7 %. The demonstration this chapter offers is therefore not that switching saves a great deal of energy on this hardware, but that the twin *quantifies the trade-off correctly*—it identifies where the saturation knee lies, prices the switching cost, bounds the attainable benefit, and exposes the energy–safety frontier that learned controllers will later be trained to navigate. A twin that reported a large saving here would be concealing the capacity mismatch rather than measuring it.

Table 1.1: Aggregate metrics over 10,090 cluster-steps (non-learning controllers; learned controllers are deferred to Chapter **??**). Energy includes switching spikes. The informed oracle sees the actual load at each step and is reported as an upper bound, not as a realisable controller.

Controller	**Energy (Wh)**	**Avg power (W)**	**Saving (%)**	**Unsafe USR (%)**	**US**
Static SD-CORE DPDK	2070.7	0.821	0.0	0.0	
Static USR-UPF	5269.8	2.089	−154.5	70.3	
Threshold	1969.6	0.780	4.9	4.2	
Hysteresis	1971.0	0.781	4.8	3.0	
Oracle (bound)	1931.0	0.765	6.7	0.0	

1.9 Summary

This chapter constructed a measurement-grounded digital twin of the UPF data plane from the profiling results of Chapter **??**. The principal points are:

1. **Scope.** The twin answers "what would happen" for a given (load, action) pair; it is neither a forecaster (Chapter **??**) nor a controller (Chapter **??**), and is kept strictly controller-agnostic.

2. **Variant choice.** `USR` is represented by `usr_full`, not `usr_safe`, precisely because a safety-aware twin must be able to predict the saturated regime; `usr_safe` would hide the failure mode and break threshold derivation.

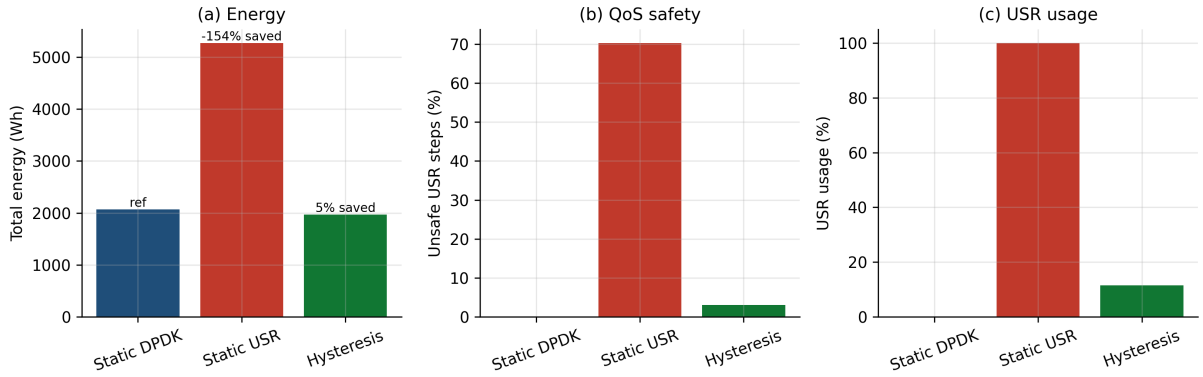

Figure 1.7: Aggregate comparison of the non-learning controllers. (a) Total energy (with saving vs. static SD-CORE DPDK annotated); (b) unsafe USR-UPF rate; (c) USR-UPF usage. Static USR-UPF is both the most expensive and by far the least safe option in this regime, since it runs a saturated data path; hysteresis achieves a modest saving while keeping the unsafe rate low.

3. **Two modes.** A stateless physics mode and a stateful rollout mode share one surrogate core; the latter adds a portable switching-cost model based on a measured activation duration and an energy-spike scaling rule.

4. **Derived, not hardcoded, thresholds.** The energy break-even and QoS limit are swept from the surrogate, and an anti-flapping band is derived from the forecaster's own error, keeping operating points consistent with the measured physics.

5. **Demonstration.** Driven by a transparent hysteresis controller, the twin recovers 4.8 % of SD-CORE DPDK's energy—roughly 72 % of the 6.7 % available to an informed oracle—while remaining safe on 97.0 % of USR-UPF steps, quantitatively exposing the energy–safety trade-off that learned controllers will later be trained to navigate.

6. **Bounded headroom.** Under the declared calibration $\alpha = 1$, the profiled USR-UPF path saturates one to two orders of magnitude below the offered cluster loads, so only $\approx 14\,\%$ of steps admit a USR-UPF decision at all and static USR-UPF costs $2.5\times$ SD-CORE DPDK. Establishing this bound is itself a result: it tells the controller chapter how much value a learned policy can possibly add, and it identifies UPF capacity—not control policy—as the dominant lever in this deployment.